

Assignment ::- 5

Name	Abhineet Tiwari
Roll No:	20965
Subject	DBMS

Q1) Create a MySQL database named *CollegeDB* and a table named *StudentResult*.

The table should contain the following columns:

Column Name	Data Type	Constraint
StudentID	INT	PRIMARY KEY
StudentName	VARCHAR(50)	
Course	VARCHAR(30)	
City	VARCHAR(30)	
Marks	INT	
FeesPaid	DECIMAL(10,2)	

Insert at least 10 records into the table.

```
CREATE DATABASE CollegeDB;
```

```
USE CollegeDB;
```

```
CREATE TABLE StudentResult (
```

```
    StudentID INT PRIMARY KEY,
```

```
    StudentName VARCHAR(50),
```

```
    Course VARCHAR(30),
```

```
    City VARCHAR(30),
```

```
    Marks INT,
```

```
    FeesPaid DECIMAL(10,2)
```

```
);
```

```
-- Insert 10 records
```

```
INSERT INTO StudentResult
```

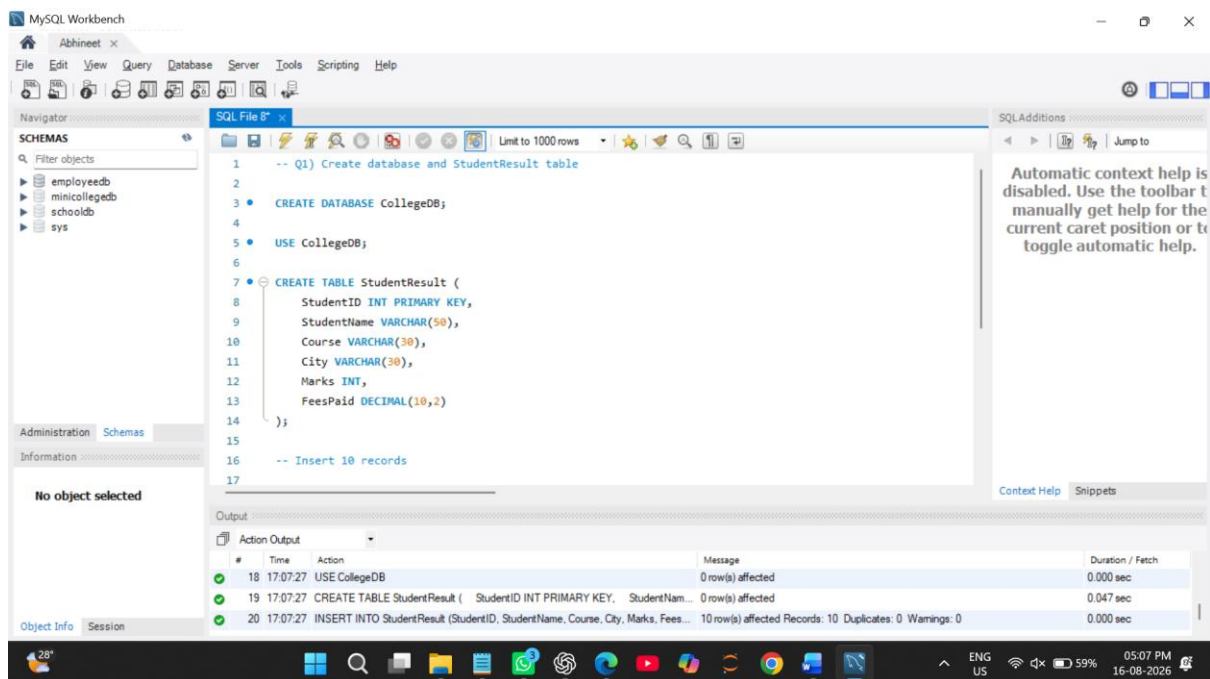
```
(StudentID, StudentName, Course, City, Marks, FeesPaid)
```

```
VALUES
```

```
(1, 'Rahul Sharma', 'BSc IT', 'Mumbai', 78, 45000.00),
```

```
(2, 'Priya Patil', 'BCA', 'Pune', 85, 40000.00),
```

(3, 'Aman Verma', 'BSc CS', 'Nashik', 67, 42000.00),
 (4, 'Sneha Joshi', 'BSc IT', 'Mumbai', 91, 45000.00),
 (5, 'Karan Mehta', 'BCA', 'Thane', 74, 40000.00),
 (6, 'Neha Shah', 'BSc IT', 'Pune', 88, 45000.00),
 (7, 'Rohan Desai', 'BSc CS', 'Mumbai', 69, 42000.00),
 (8, 'Meera Kulkarni', 'BCA', 'Nashik', 82, 40000.00),
 (9, 'Arjun Nair', 'BSc IT', 'Thane', 59, 45000.00),
 (10, 'Divya More', 'BCA', 'Mumbai', 93, 40000.00);



Q2) Write MySQL queries to find the following from the StudentResult table:

Requirement
Total number of students
Total fees paid
Average marks
Lowest marks
Highest marks

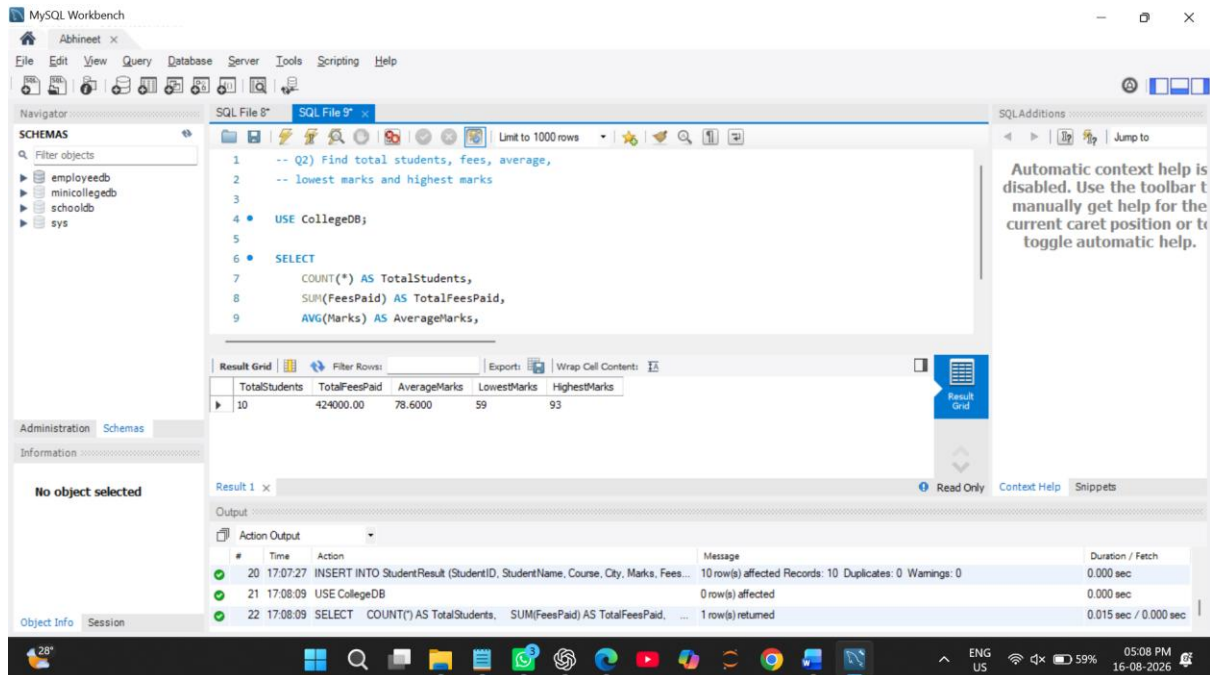
-- lowest marks and highest marks

USE CollegeDB;

SELECT

COUNT(*) AS TotalStudents,
SUM(FeesPaid) AS TotalFeesPaid,
AVG(Marks) AS AverageMarks,
MIN(Marks) AS LowestMarks,
MAX(Marks) AS HighestMarks

FROM StudentResult;



Q3) Write a MySQL query to count the number of students in each course.

The output should contain:

Column
Course
TotalStudents

USE CollegeDB;

SELECT

Course,

COUNT(*) AS TotalStudents

FROM StudentResult

GROUP BY Course;

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
-- Q3) Count students in each course
USE CollegeDB;

SELECT
    Course,
    COUNT(*) AS TotalStudents
FROM StudentResult
GROUP BY Course;
```

The Results window displays the following data:

Course	TotalStudents
BSc IT	4
BCA	4
BSc CS	2

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
22	17:08:09	SELECT	COUNT(*) AS TotalStudents, SUM(FeesPaid) AS TotalFeesPaid, ...	1 row(s) returned 0.015 sec / 0.000 sec
23	17:08:44	USE	CollegeDB	0 row(s) affected 0.000 sec
24	17:08:44	SELECT	Course, COUNT(*) AS TotalStudents FROM StudentResult GROUP ...	3 row(s) returned 0.000 sec / 0.000 sec

Q4) Write a MySQL query to find the average marks of students course-wise.

The output should contain:

Column
Course
AverageMarks

USE CollegeDB;

SELECT

Course,

AVG(Marks) AS AverageMarks

FROM StudentResult

GROUP BY Course;

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
-- Q4) Find average marks course-wise  
  
USE CollegeDB;  
  
SELECT  
    Course,  
    AVG(Marks) AS AverageMarks  
FROM StudentResult  
GROUP BY Course;
```

The Results window displays the following data:

Course	AverageMarks
BSc IT	79.0000
BCA	83.5000
BSc CS	68.0000

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
24	17:08:44	SELECT Course, COUNT(*) AS TotalStudents FROM StudentResult GROUP BY Course	3 row(s) returned	0.000 sec / 0.000 sec
25	17:09:11	USE CollegeDB	0 row(s) affected	0.016 sec
26	17:09:11	SELECT Course, AVG(Marks) AS AverageMarks FROM StudentResult GROUP BY Course	3 row(s) returned	0.015 sec / 0.000 sec

Q5) Write a MySQL query to display only those courses where the number of students is greater than 2.

Use:

Clause
GROUP BY
HAVING

USE CollegeDB;

SELECT

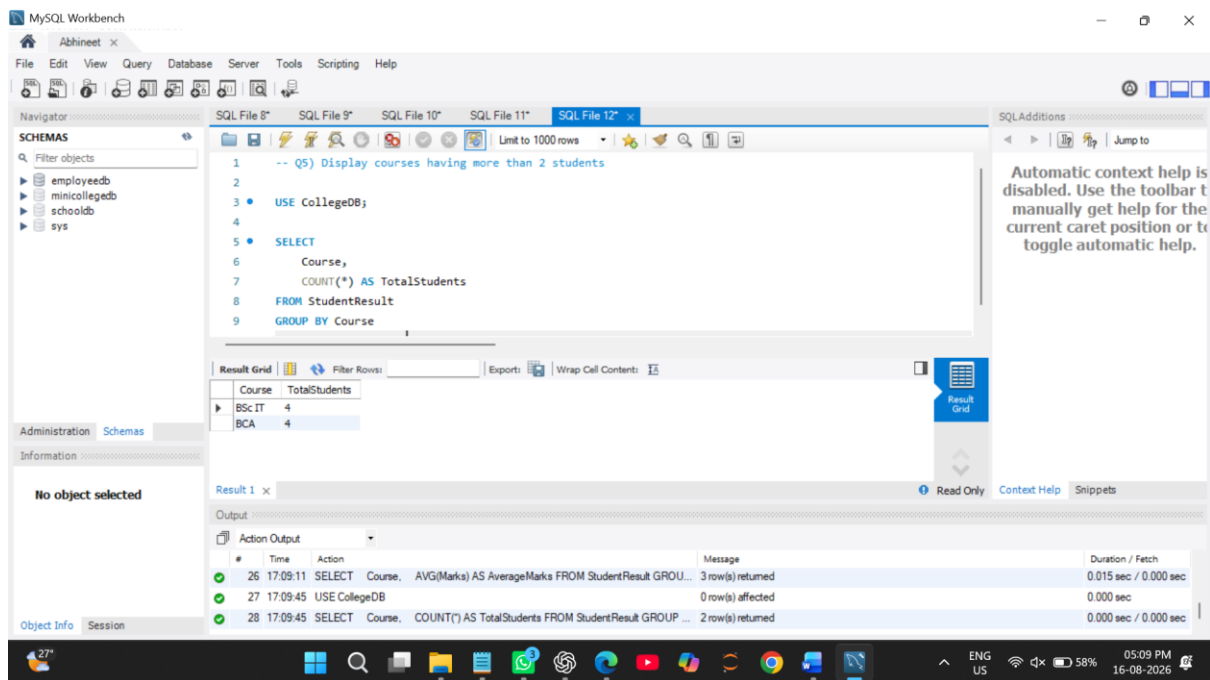
Course,

COUNT(*) AS TotalStudents

FROM StudentResult

GROUP BY Course

HAVING COUNT(*) > 2;



Q6) Write a MySQL query to analyze course performance.

The output should display:

Column
Course
TotalStudents
AverageMarks
LowestMarks
HighestMarks

Sort the result by **AverageMarks** in descending order.

USE CollegeDB;

SELECT

Course,

COUNT(*) AS TotalStudents,

AVG(Marks) AS AverageMarks,

MIN(Marks) AS LowestMarks,

MAX(Marks) AS HighestMarks

FROM StudentResult

GROUP BY Course

ORDER BY AverageMarks DESC;

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
-- Q6) Analyze course performance
USE CollegeDB;

SELECT
  Course,
  COUNT(*) AS TotalStudents,
  AVG(Marks) AS AverageMarks,
  MIN(Marks) AS LowestMarks,
```

The results are displayed in the Result Grid:

Course	TotalStudents	AverageMarks	LowestMarks	HighestMarks
BCA	4	83.5000	74	93
BSc IT	4	79.0000	59	91
BSc CS	2	68.0000	67	69

The bottom panel shows the Action Output with the following log:

#	Time	Action	Message	Duration / Fetch
28	17:09:45	SELECT Course, COUNT(*) AS TotalStudents FROM StudentResult GROUP ...	2 row(s) returned	0.000 sec / 0.000 sec
29	17:10:13	USE CollegeDB	0 row(s) affected	0.000 sec
30	17:10:13	SELECT Course, COUNT(*) AS TotalStudents, AVG(Marks) AS AverageMa...	3 row(s) returned	0.000 sec / 0.000 sec